

Python *args and **kwargs

25 Coding Questions — Basic to Medium

Instructions

- Every question asks you to **write working Python code**. Save each solution as a separate file such as q01.py, q02.py, and so on.
- Read the task, then the requirements, then study the example. Test your function with the example inputs **and** with your own edge cases (no arguments, one argument, negative numbers, and so on).
- **Rule on function calls:** inside your solutions, one user-defined function must not call another user-defined function. Each question is solved by a single function. Calling Python built-ins such as print(), len(), type(), range(), enumerate() and round() is fine. **return** statements are expected. Nested functions (defining a function inside a function) appear only in Question 24.
- Wherever the question says 'return', the function should return the value and NOT print it. Printing is done in the main program that calls your function.
- Add a short comment above each function describing what it does.

Quick Reference

Syntax	Meaning	Stored as
*args	Collects extra positional arguments	tuple
**kwargs	Collects extra keyword (name=value) arguments	dict
def f(a, b=1, *args, **kwargs)	Required order of parameters	-
def f(*args, key=1)	key becomes keyword-only	-
f(*my_list)	Unpacks a list/tuple into positional arguments	-
f(**my_dict)	Unpacks a dictionary into keyword arguments	-

Question Map

Q1–Q10: *args **Q11–Q17:** **kwargs **Q18–Q25:** *args + **kwargs combined

Part A: *args

Q1. Sum of Any Number of Values [Basic]

Write a function that accepts any number of numeric arguments and returns their total. The function must work whether the caller passes zero, one, or many numbers.

Function signature

```
def add_all(*args):
```

What your code must do

- Create a variable total = 0 and add each value from args to it using a for loop (do not use the built-in sum()).
- Return the final total (do not print inside the function).
- If no arguments are passed, the function must return 0.

Example

```
add_all() -> 0
add_all(5) -> 5
add_all(2, 4, 6, 8) -> 20
```

Q2. Average Calculator [Basic]

Create a function that calculates the average (mean) of all numbers given to it. You must first count how many numbers were received and then divide the total by that count.

Function signature

```
def average(*args):
```

What your code must do

- Use len(args) to find how many numbers were passed.
- If len(args) is 0, return 0 immediately (this is called an early return).
- Otherwise compute the total with a loop and return total / count.

Example

```
average(10, 20, 30) -> 20.0
average(7, 8) -> 7.5
average() -> 0
```

Q3. Find the Largest Number (Without max()) [Basic]

Write a function that returns the largest value among all arguments, but without using the built-in max() function. This practises loops, comparison and tracking a 'best so far' variable.

Function signature

```
def find_largest(*args):
```

What your code must do

- If args is empty, return None.
- Start by assuming the first argument args[0] is the largest and store it in a variable called largest.
- Loop through every value; whenever a value is bigger than largest, update largest.

Example

```
find_largest(3, 9, 2, 7) -> 9
find_largest(-5, -2, -9) -> -2
find_largest() -> None
```

Q4. Count and Inspect the Arguments [Basic]

Write a function to understand what `*args` really is. The function should receive any number of values and report how many were passed and what type of container Python used to store them.

Function signature

```
def inspect_args(*args):
```

What your code must do

- Print the line: Number of arguments: `<count>`.
- Print the line: Type of args: `<type>` using `type(args)`.
- Print every argument on its own line together with its index (use `enumerate`).

Example

```
inspect_args('a', 10, 3.5)
Number of arguments: 3
Type of args: <class 'tuple'>
0 -> a
1 -> 10
2 -> 3.5
```

Q5. Product of Numbers [Basic]

Write a function that multiplies all the numbers passed to it and returns the result. Be careful with the starting value of your accumulator: for multiplication the neutral starting value is 1, not 0.

Function signature

```
def multiply_all(*args):
```

What your code must do

- Start with `result = 1`.
- Multiply result by each value in `args` using a loop.
- If no arguments are given, return 1 (the empty product by convention).

Example

```
multiply_all(2, 3, 4) -> 24
multiply_all(5, 0, 8) -> 0
multiply_all(-2, 6) -> -12
```

Q6. Join Words with a Separator [Basic]

Build a function that joins any number of words into one sentence using a separator chosen by the caller. The separator must be a keyword-only argument: a parameter that comes after `*args` can only be supplied by name, such as `sep=' '`.

Function signature

```
def join_words(*args, sep=' '):
```

What your code must do

- Loop through `args` and build a result string, inserting `sep` between words but NOT after the last word.
- Do not use `str.join()`; write the logic yourself with a loop and an index or a first-item check.
- Return the final string. If no words are passed, return an empty string `''`.

Example

```
join_words('Python', 'is', 'fun') -> 'Python is fun'
join_words('2025', '01', '15', sep='-') -> '2025-01-15'
```

Q7. Greet Multiple People [Basic]

Write a function with one normal (required) parameter followed by *args. The first parameter is the greeting word, and *args holds any number of names.

Function signature

```
def greet_all(greeting, *args):
```

What your code must do

- For each name build the string: '<greeting>, <name>!'.
• Collect all messages in a list and return the list.
• If no names are given, return an empty list.

Example

```
greet_all('Hello', 'Asha', 'Ravi') -> ['Hello, Asha!', 'Hello, Ravi!']  
greet_all('Hi') -> []
```

Q8. Positive, Negative and Zero Counter [Basic]

Write a function that takes any number of integers and counts how many are positive, how many are negative and how many are zero. Return all three counts together.

Function signature

```
def count_signs(*args):
```

What your code must do

- Use three counters: positive, negative, zero.
• Use if / elif / else inside a loop to update the correct counter.
• Return the three counts as: return positive, negative, zero.

Example

```
count_signs(4, -1, 0, 7, -3) -> (2, 2, 1)
```

Q9. Filter Even Numbers [Basic]

Create a function that receives any number of integers and returns a new list containing only the even numbers, in the same order in which they were received. The original arguments must not be modified.

Function signature

```
def only_even(*args):
```

What your code must do

- Create an empty list called result.
• Loop through args; if value % 2 == 0, append it to result.
• Return result (a list, not a tuple).

Example

```
only_even(1, 2, 3, 4, 6) -> [2, 4, 6]  
only_even(1, 3, 5) -> []
```

Q10. Reverse the Arguments [Basic]

Write a function that returns the arguments in reverse order as a tuple. You may NOT use slicing `[::-1]` or `reversed()` for this question.

Function signature

```
def reverse_args(*args):
```

What your code must do

- Create an empty list called `items`.
- Loop with an index from `len(args) - 1` down to 0 (use `range(len(args) - 1, -1, -1)`).
- Append `args[index]` to `items` on each pass.

Example

```
reverse_args(1, 2, 3) -> (3, 2, 1)
reverse_args('a') -> ('a',)
reverse_args() -> ()
```

Part B: **kwargs

Q11. Print a Profile Card [Basic]

Write a function that accepts any number of keyword arguments (name=value pairs) and prints them as a neat profile card. `**kwargs` collects all keyword arguments into a dictionary, so you can loop over it with `.items()`.

Function signature

```
def show_profile(**kwargs):
```

What your code must do

- First print a header line: `===== PROFILE =====`.
- Loop through `kwargs.items()` and print each pair as: `<Key in Title Case>: <value>` (use `.title()` on the key and replace underscores with spaces).
- Print a footer line of the same length.

Example

```
show_profile(name='Asha', favourite_color='Blue', age=21)
===== PROFILE =====
Name: Asha
Favourite Color: Blue
Age: 21
```

Q12. Count and List the Keywords [Basic]

Write a function that tells the caller how many keyword arguments were given and returns a list of their names. This shows that `kwargs` is a normal dictionary and that its keys are the argument names written as strings.

Function signature

```
def keyword_info(**kwargs):
```

What your code must do

- Print Type of `kwargs`: `<type>` using `type(kwargs)`.
- Build a list of the keys (use a loop, not `list(kwargs)`).
- Return a tuple: `(number_of_keywords, list_of_keys)`.

Example

```
keyword_info(a=1, b=2) -> (2, ['a', 'b'])
keyword_info() -> (0, [])
```

Q13. Total Marks Calculator [Basic]

A teacher passes the marks of a student as keyword arguments where the name is the subject and the value is the mark. Write a function that calculates and returns the total marks and the percentage assuming each subject is out of 100.

Function signature

```
def total_marks(**kwargs):
```

What your code must do

- Loop through `kwargs.values()` and add up all marks.
- Compute `percentage = total / (number_of_subjects * 100) * 100`. Guard against zero subjects by returning `(0, 0.0)`.
- Return both values as a tuple `(total, percentage)` with percentage rounded to 2 decimals.

Example

```
total_marks(maths=90, science=80, english=70) -> (240, 80.0)
```

Q14. Uppercase Keys Dictionary [Basic]

Write a function that receives keyword arguments and returns a NEW dictionary in which every key has been converted to uppercase while the values stay unchanged. The function must not change the incoming `kwargs` dictionary.

Function signature

```
def upper_keys(**kwargs):
```

What your code must do

- Create an empty dictionary called `result`.
- Loop through `kwargs.items()` and add `result[key.upper()] = value`.
- Return `result`.

Example

```
upper_keys(city='Jaipur', pin=302001) -> {'CITY': 'Jaipur', 'PIN': 302001}
```

Q15. Find Keys by Value [Basic]

Write a function that searches keyword arguments for a given value and returns the list of all keyword names that have that value. The search value is a normal positional parameter placed before `**kwargs`.

Function signature

```
def keys_with_value(target, **kwargs):
```

What your code must do

- Loop through `kwargs.items()` and compare each value with `target`.
- Append the matching key names to a list.
- Return the list (empty list if nothing matches).

Example

```
keys_with_value(10, a=10, b=20, c=10) -> ['a', 'c']  
keys_with_value(99, a=1) -> []
```

Q16. Pass-Mark Filter [Basic]

Write a function that receives a pass mark and keyword arguments of subject marks. Return a dictionary containing only the subjects in which the student scored at least the pass mark.

Function signature

```
def passed_subjects(pass_mark, **kwargs):
```

What your code must do

- Create an empty dictionary named passed.
- Loop through kwargs.items(); if the mark is $\geq$ pass_mark, store it in passed.
- Return passed.

Example

```
passed_subjects(40, maths=75, physics=35, chemistry=40) -> {'maths': 75, 'chemistry': 40}
```

Q17. Merge Default Settings with Overrides [Medium]

Many programs have default settings that the user may override. Write a function that takes a dictionary of default settings as its first parameter and any number of keyword overrides.

Function signature

```
def merge_settings(defaults, **kwargs):
```

What your code must do

- Make a copy of defaults first (result = defaults.copy()) so the original dictionary is not modified.
- Loop through kwargs.items() and set result[key] = value.
- Return result.
- Prove the original is untouched by printing defaults before and after the call.

Example

```
base = {'theme': 'light', 'font': 12}
merge_settings(base, theme='dark', language='en')
-> {'theme': 'dark', 'font': 12, 'language': 'en'}
base is still {'theme': 'light', 'font': 12}
```

Hint: If you skip copy(), both variables point to the same dictionary, and changing one changes the other.

Part C: Combining *args and **kwargs

Q18. Understand the Parameter Order [Medium]

Python has a strict order for parameters: normal parameters, then default parameters, then *args, then **kwargs. Write a function that follows this exact order and returns a dictionary showing what each part received.

Function signature

```
def show_order(a, b=10, *args, **kwargs):
```

What your code must do

- Return a dictionary with four keys: 'a', 'b', 'args' and 'kwargs', holding the values received.
- args must be stored as a list (use list(args) or a loop) and kwargs as a dictionary.
- In the main program, call the function in three ways: show_order(1), show_order(1, 2, 3, 4) and show_order(1, 2, 3, x=5, y=6). Print each result.
- Write a comment above each call explaining which value went to which parameter.

Example

```
show_order(1, 2, 3, x=5) -> {'a': 1, 'b': 2, 'args': [3], 'kwargs': {'x': 5}}
```

Hint: Extra positional values go to args only AFTER a and b are filled.

Q19. Shopping Bill with Discount and Fees [Medium]

A shop passes the item prices as positional arguments, an optional discount percentage as a keyword-only argument, and any extra charges (for example delivery=50, packing=20) as keyword arguments. Calculate the final bill amount.

Function signature

```
def final_bill(*prices, discount=0, **fees):
```

What your code must do

- Calculate the subtotal by adding all prices.
- Calculate discount_amount = subtotal * discount / 100 and subtract it.
- Add every value from fees.values() to the amount.
- Return the final amount rounded to 2 decimals; print an itemised summary in the main program (subtotal, discount, each fee, total).

Example

```
final_bill(500, 250, discount=10, delivery=40, packing=10) -> 725.0  
(750 - 75 + 40 + 10 = 725)
```

Hint: discount must be passed by name because it comes after *prices. Any other name=value pair falls into fees automatically.

Q20. Calling with Unpacking (* and **) [Medium]

The * and ** symbols also work when CALLING a function: * unpacks a list or tuple into positional arguments and ** unpacks a dictionary into keyword arguments. First write a normal function, then call it three different ways to see unpacking in action.

Function signature

```
def describe_person(name, age, city):
```

What your code must do

- The function returns the string: '<name> is <age> years old and lives in <city>.'
- Call 1: pass values directly describe_person('Asha', 21, 'Jaipur').
- Call 2: store values in a list data = ['Ravi', 25, 'Delhi'] and call describe_person(*data).
- Call 3: store values in a dictionary info = {'name': 'Meera', 'age': 30, 'city': 'Pune'} and call describe_person(**info).

Example

All three calls return a sentence like: 'Asha is 21 years old and lives in Jaipur.'

Hint: If the dictionary has a key that is not a parameter name, Python raises a TypeError. Try it and read the error message.

Q21. Student Report Generator [Medium]

Write a function that creates a report for a student. The name is a required parameter, the marks are any number of positional values (*scores) and extra information such as roll_no, class or city is given as keywords (**details).

Function signature

```
def student_report(name, *scores, **details):
```

What your code must do

- Calculate total and average of scores (handle the case with no scores; average = 0).
- Decide a grade: average >= 90 'A', >= 75 'B', >= 50 'C', otherwise 'D'.
- Build the result dictionary with keys: name, total, average, grade, and then add all items of details into it using a loop.
- Print the report in the main program one key per line.

Example

```
student_report('Asha', 90, 85, 95, roll_no=12, city='Jaipur')  
-> {'name': 'Asha', 'total': 270, 'average': 90.0, 'grade': 'A', 'roll_no': 12, 'city': 'Jaipur'}
```

Hint: Write the grade logic with if / elif / else and keep it directly inside the function.

Q22. Flexible Calculator [Medium]

Create a calculator that applies an operation to any quantity of numbers. The first argument is a string naming the operation: 'add', 'multiply', 'subtract' or 'max'.

Function signature

```
def calculate(operation, *numbers):
```

What your code must do

- Use an if / elif chain on operation to select the logic; write each logic with a loop (no built-in sum, max or math functions).
- If numbers is empty, return None.
- If the operation is not recognised, return the string 'Unknown operation'.
- Test each operation, an empty call such as calculate('add') and an invalid call such as calculate('divide', 4, 2).

Example

```
calculate('add', 1, 2, 3) -> 6  
calculate('subtract', 20, 5, 3) -> 12  
calculate('max', 4, 9, 2) -> 9
```

Hint: For subtract: start result = numbers[0], then loop over numbers[1:] and subtract each value.

Q23. HTML Tag Builder [Medium]

Write a function that builds an HTML element as a string. The tag name is required, the content pieces are positional (*content) and the HTML attributes such as id or class_name are keywords (**attributes).

Function signature

```
def make_tag(tag, *content, **attributes):
```

What your code must do

- Build the attribute text by looping over attributes.items() producing key="value" pairs separated by spaces; replace the key class_name with class.
- Join all content pieces together, with a single space between pieces.
- Return the string: <tag attributes>content</tag>. If there are no attributes, do not leave an extra space inside the opening tag.
- Test with and without attributes and with several content pieces.

Example

```
make_tag('p', 'Hello', 'World') -> <p>Hello World</p>
```

```
make_tag('a', 'Click here', href='site.com', class_name='btn') -> <a href="site.com" class="btn">Click here</a>
```

Hint: Build the attribute string first (it can be empty), then decide whether to add a leading space before it.

Q24. Power-Sum Closure (Nested Function) [Medium]

Nested functions are allowed in this assignment. Write an outer function that receives any number of exponents through *args and returns an inner function.

Function signature

```
def make_power_sum(*exponents):  
    def inner(*numbers):  
        ...  
    return inner
```

What your code must do

- The outer function must return the inner function itself without calling it (return inner, not return inner()).
- Inside inner, use two loops: for each number and for each exponent, add number ** exponent to a total.
- Return the total from inner.
- In the main program: f = make_power_sum(2, 3); print(f(1, 2)). Then create another closure with g = make_power_sum(1) and show it behaves differently.

Example

```
f = make_power_sum(2, 3)
```

```
f(1, 2) -> (1**2 + 1**3) + (2**2 + 2**3) = 2 + 12 = 14
```

Hint: inner can read exponents because it was defined inside make_power_sum. Each call to make_power_sum creates a separate remembered exponents tuple.

Q25. User Registration with Keyword-Only Rules [Medium]

Write a function that registers a user. The username is the only normal positional parameter.

Function signature

```
def register_user(username, *, role='guest', **extra):
```

What your code must do

- If username is shorter than 3 characters, return the dictionary {'error': 'Username too short'}.
- Only the roles 'guest', 'member' and 'admin' are valid; otherwise return {'error': 'Invalid role'}.
- On success return {'username': ..., 'role': ..., 'extra': extra}.
- Test valid input, a short username, an invalid role, and the call register_user('asha', 'admin') which must raise a TypeError because role is keyword-only. Explain the error in a comment.

Example

```
register_user('asha', role='admin', email='a@x.com', age=21)
-> {'username': 'asha', 'role': 'admin', 'extra': {'email': 'a@x.com', 'age': 21}}
```

Hint: A bare * in a def line is not a parameter itself; it is only a marker that says everything after it must be passed by name.